

template

目录

1. 数论	2
1.1 素数	2
1.1.1 线性筛素数	2
1.1.2 埃筛素数	2
1.1.3.1 素性检验（试除法）	2
1.1.3.2 素性检验（Miller-Rabin）	2
1.1.4 质因数分解	3
1.2 欧拉函数	3
1.2.1 根号欧拉函数	3
1.2.2 欧拉求和	4
1.2.2 欧拉求和与反演	4
1.3 筛法	4
1.3.1 筛法求欧拉函数	4
1.3.2 筛法求莫比乌斯函数	5
1.3.3 筛法求约数个数函数	5
1.3.4 筛法求约数和	5
1.4 最大公约数	6
1.4.1 欧几里得算法	6
1.4.2 扩展欧几里得算法	6
1.5 模算术类	6
1.5.1 快速幂	6
1.5.2 逆元	6
1.5.3 卢卡斯定理	7
1.5.4 扩展卢卡斯定理	7
1.6 中国剩余定理	7
1.6.1 CRT（模数互质）	7
1.6.2 exCRT（模数不互质）	7
1.7 数论分块	8
2. 线性代数	8
2.1 高斯消元	8

2.2 矩阵快速幂	9
2.3 线性基	9
2.4 行列式	9
2.5 矩阵求逆	10
2.6 递推转矩阵	10
3. 组合数学	10
3.1 组合数预处理	10
3.2 组合数取模（阶乘）	11
3.3 容斥原理	11
3.4 Catalan 数	11
3.5 排列与错排	11
3.6 Stirling 数	11
3.7 Bell 数	12
3.8 多重集与插板法	12
3.9 Burnside 引理	12
3.10 Pólya 定理	12
4. 多项式与生成函数	12
4.1 NTT	12
4.1.1 NTT 半家桶 ($\ln/\exp/\text{pow}/\text{sqrt}$)	13
4.2 多项式求逆	15
4.3 多项式求导与积分	15
4.4 生成函数常见结论	15
4.5 FWT（按位卷积）	16
4.5.1 异或卷积	16
4.5.2 或卷积	16
4.5.3 与卷积	16
4.6 拉格朗日插值	16
4.7 Newton 迭代（多项式）	17
4.8 生成函数常用模型	17
5. 数值算法	17
5.1 三分法	17
5.2 Simpson 积分	17
5.3 牛顿迭代	18
5.4 二分答案	18
5.5 牛顿迭代求平方根	18
5.6 黄金分割	18
6. 博弈论	18
6.1 Nim 游戏	18

6.2 SG 函数	19
6.3 DAG 博弈	19
6.4 威佐夫博弈	19
6.5 斐波那契尼姆博弈	19
6.6 mex 性质	20
6.7 记忆化模板	20
7. 计算几何	20
7.1 计算几何基础模板	20
7.2 凸包相关	23
7.2.1 旋转卡壳求凸包直径	23
7.2.2 Minkowski 和（凸多边形）	23
7.3 圆与直线	23
7.4 最近点对（分治）	24
8. 杂项	25
8.1 高精度全家桶	25
8.2 快速平方根取倒数	27
9. 数据结构	27
9.1 FHQ Treap	27
9.2 CDQ 分治	29
9.3 ST 表	29
9.4 树链剖分	29
9.5 树上倍增 / RMQ LCA（原模板）	30
10. 图论	30
10.1 最短路（Dijkstra）	30
10.2 Dinic 最大流	31
10.3 费用流（MCMF）	32
10.4 Tarjan 强连通分量	33

1. 数论

1.1 素数

1.1.1 线性筛素数

```
// bool vis[N]
void getprime(int MAXN){
    for(int i=2;i<=MAXN;i++){
        if(!vis[i]) prim[++prim[0]]=i;
        for(int j=1;j<=prim[0]&&prim[j]*i<=MAXN;++j){
            vis[prim[j]*i]=1;
            if(i%prim[j]==0) break;
        }
    }
}
// int vis[N]
void getprim(int MAXN){
    for(int i=2;i<=MAXN;++i){
        if(!vis[i]) vis[i]=prim[++prim[0]]=i;
        for(int j=1;j<=prim[0]&&i*prim[j]<=MAXN;++j){
            vis[i*prim[j]]=prim[j];
        }
    }
}
```

1.1.2 埃筛素数

```
void getprim(int MAXN){
    int cnt(0);
    for(int i=2;i<=MAXN;++i){
        if(!vis[i]){
            prim[++cnt]=i;
            for(int j=i+i;j<=MAXN;j+=i) vis[j]=1;
        }
    }
}
```

1.1.3.1 素性检验（试除法）

```
bool isPrime(int a){
    if(a<2) return 0;
    for(int i=2;i<a;++i)
        if(a%i==0) return 0;
    return 1;
}
```

1.1.3.2 素性检验（Miller-Rabin）

二次探测定理：若 p 为奇素数，则 $x^2 \equiv 1 \pmod{p}$ 或者 $x^2 \equiv p-1 \pmod{p}$.

```
bool MillerRabin(int n){
    int test_time(30);
```

```

    if(n<3||n%2==0) return n==2;
    if(n%3==0) return n==3;
    int u(n-1),t(0);
    for(;u%2==0;u>>=1,++t);
    for(;test_time-->0){
        int a(rand()%(n-3)+2);
        int v(qpow(a,u,n)); // 快速幂
        if(v==1) continue;
        int s;
        for(s=0;s<t;++s){
            if(v==n-1) break;
            v=(long long) v*v%n;
        }
        if(s==t) return 0;
    }
    return 1;
}

```

1.1.4 质因数分解

```

vector<int> BreakDown(int MAXN){
    vector<int> res;
    for(int i=2;i*i<=MAXN;++i){
        if(MAXN%i==0){
            for(;MAXN%i==0;MAXN/=i);
            res.push_back(i);
        }
    }
    if(MAXN>1) res.push_back(MAXN);
    return res;
}

```

1.2 欧拉函数

1.2.1 根号欧拉函数

设 $n = \prod_{i=1}^s p_i^{k_i}$ ，则有：

$$\begin{aligned}
 \varphi(n) &= \prod_{i=1}^s \varphi(p_i^{k_i}) \\
 &= \prod_{i=1}^s (p_i - 1) p_i^{k_i - 1} \\
 &= \prod_{i=1}^s \left(1 - \frac{1}{p_i}\right) p_i^{k_i} \\
 &= n \prod_{i=1}^s \left(1 - \frac{1}{p_i}\right)
 \end{aligned}$$

```

int euler_phi(int n){
    int ans(n);
    for(int i=2;i*i<=n;i++){
        if(n%i==0){
            ans=ans/i*(i-1);
        }
    }
    return ans;
}

```

```

        for(n%i==0) n/=i;
    }
    if(n>1) ans=ans/n*(n-1);
    return ans;
}

```

推论：对任意不全为 0 的整数 m, n , $\varphi(mn)\varphi(\gcd(m, n)) = \varphi(m)\varphi(n)\gcd(m, n)$.

1.2.2 欧拉求和

$$n = \sum_{d|n} \varphi(d)$$

1.2.2 欧拉求和与反演

$$\begin{aligned}
 n &= \sum_{d|n} \varphi(d) \\
 \gcd(a, b) &= \sum_{d|\gcd(a, b)} \varphi(d) \\
 \gcd(a, b) &= \sum_d [d|a][d|b]\varphi(d) \\
 \sum_{i=1}^n \gcd(i, n) &= \sum_{d|n} \lfloor \frac{n}{d} \rfloor \varphi(d)
 \end{aligned}$$

常用莫比乌斯反演：若 $f(n) = \sum_{d|n} g(d)$, 则

$$g(n) = \sum_{d|n} \mu(d) f\left(\frac{n}{d}\right)$$

以及

$$\sum_{d|n} \mu(d) = [n = 1]$$

1.3 筛法

1.3.1 筛法求欧拉函数

```

void pre_eular(int MAXN){
    phi[1]=1;
    for(int i=2;i<=MAXN;++i){
        if(!phi[i]) prim[++prim[0]]=i, phi[i]=i-1;
        for(int j=1;j<=prim[0]&& i*prim[j]<=MAXN;++j){
            if(!(i%prim[j])){
                phi[i*prim[j]]=prim[j]*phi[i];
                break;
            }
            phi[i*prim[j]]=(prim[j]-1)*phi[i];
        }
    }
}

```

1.3.2 筛法求莫比乌斯函数

```
void pre_mu(int MAXN){
    mu[1]=1;
    for(int i=2;i<=MAXN;i++){
        if(!vis[i]) prime[++cnt]=i,mu[i]=-1;
        for(int j=1;j<=cnt&& i*prime[j]<=MAXN;j++){
            vis[i*prime[j]]=1;
            if(i%prime[j]==0) break;
            mu[i*prime[j]]=-mu[i];
        }
    }
}
```

1.3.3 筛法求约数个数函数

```
void pre_d(int MAXN){
    d[1]=1;
    for(int i=2;i<=MAXN;++i){
        if(!vis[i]) prim[++prim[0]]=i,d[i]=2,num[i]=1;
        for(int j=1;j<=prim[0]&& i*prim[j]<=MAXN;++j){
            vis[prim[j]*i]=1;
            if(i%prim[j]==0){
                num[i*prim[j]]=num[i]+1;
                d[i*prim[j]]=d[i]/(num[i]+1)*(num[i]+2);
                break;
            }
            num[i*prim[j]]=1;
            d[i*prim[j]]=d[i]<<1;
        }
    }
}
```

1.3.4 筛法求约数和

生成函数乘积，取质因子随机组合。

```
void pre_f(int MAXN){
    // g[i] 为 i 的最小质因子 p 的 k 次定比数列和
    g[1]=f[1]=1;
    for(int i=2;i<=MAXN;++i){
        if(!vis[i]) prim[++prim[0]]=i,g[i]=f[i]=i+1;
        for(int j=1;j<=prim[0]&& i*prim[j]<=MAXN;++j){
            vis[prim[j]*i]=1;
            if(i%prim[j]==0){
                g[i*prim[j]]=g[i]*prim[j]+1;
                f[i*prim[j]]=f[i]/g[i]*g[i*prim[j]];
                break;
            }
            g[i*prim[j]]=1+prim[j];
            f[i*prim[j]]=f[i]*(prim[j]+1);
        }
    }
}
```

1.4 最大公约数

1.4.1 欧几里得算法

```
int gcd(int a,int b){return !b?a:gcd(b,a%b);}
int gcd(int a,int b){
    for(int tmp;b;)
        tmp=a,a=b,b=tmp%b;
    return a;
}
```

1.4.2 扩展欧几里得算法

```
void Exgcd(ll a,ll b,ll &x,ll &y){
    if(!b) x=1,y=0;
    else Exgcd(b,a%b,y,x),y-=a/b*x;
}
```

1.5 模算术类

1.5.1 快速幂

```
ll qpow(ll a,ll b,ll p){
    ll res(1);
    for(;b;b>>=1,a=a*a%p) if(b&1) res=res*a%p;
    return res%p;
}
```

1.5.2 逆元

1.5.2.1 费马小定理

```
int inv=qpow(a,p-2,p);// p 为质数
```

1.5.2.2 线性求逆元

$$i^{-1} \equiv -\left\lfloor \frac{p}{i} \right\rfloor \times (p \bmod i)^{-1} \pmod{p}$$

```
inv[1]=1;
for(int i=2;i<p;++i)
    inv[i]=(p-p/i)*inv[p%i]%p;
```

1.5.2.2 扩欧求逆元

```
void Exgcd(ll a,ll b,ll &x,ll &y){
    if(!b) x=1,y=0;
    else Exgcd(b,a%b,y,x),y-=a/b*x;
}
int main(){
```

```

    ll x,y;
    Exgcd(a,p,x,y);
    x=(x%p+p)%p;//x 是 a 在 mod p 下的逆元
}

```

1.5.2.3 阶乘逆元

$$\frac{1}{i!} \times i = \frac{1}{(i-1)!}$$

得到：

$$\frac{1}{i!} \times (i-1)! = \frac{1}{i}$$

更一般的将 i 替换为 a_i 也可线性求列表逆元。

1.5.3 卢卡斯定理

```

ll Lucas(ll n,ll k,ll p) {
    if(!k) return 1ll;
    return (C(n%p,k%p)*Lucas(n/p,k/p,p))%p;
} // p 为质数

```

1.5.4 扩展卢卡斯定理

1.6 中国剩余定理

1.6.1 CRT（模数互质）

若 m_i 两两互质，求

$$x \equiv a_i \pmod{m_i}$$

则唯一解模 $M = \prod m_i$ 。

```

ll CRT(vector<ll> a,vector<ll> m){
    ll M=1,ans=0;
    for(auto x:m) M*=x;
    for(int i=0;i<(int)a.size();++i){
        ll Mi=M/m[i],x,y;
        Exgcd(Mi,m[i],x,y);
        x=(x%m[i]+m[i])%m[i];
        ans=(ans+a[i]*Mi%M*x)%M;
    }
    return (ans%M+M)%M;
}

```

1.6.2 exCRT（模数不互质）

逐个合并：

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \end{cases}$$

若 $a_2 - a_1$ 不能被 $\gcd(m_1, m_2)$ 整除, 则无解。

```
bool exCRT(ll a1, ll m1, ll a2, ll m2, ll &a, ll &m){
    ll x, y;
    ll g = gcd(m1, m2);
    if((a2 - a1) % g) return 0;
    Exgcd(m1, m2, x, y);
    ll p = m2 / g;
    x = (x % p + p) % p;
    ll k = ((a2 - a1) / g % p + p) % p * x % p;
    a = a1 + m1 * k;
    m = m1 / g * m2;
    a = (a % m + m) % m;
    return 1;
}

bool merge(const vector<ll> &a, const vector<ll> &m, ll &a_, ll &m_){
    if(a.empty()) return 0;
    a_ = a[0], m_ = m[0];
    for(int i = 1; i < (int)a.size(); ++i)
        if(!exCRT(a)) return 0;
    return 1;
}
```

1.7 数论分块

数论分块常用于处理 $\sum_{i=1}^n \lfloor \frac{n}{i} \rfloor$ 、 $\sum f(\lfloor n/i \rfloor)$ 之类的式子。

```
for(ll l = 1, r; l <= n; l = r + 1){
    r = n / (n / l);
    // [l, r] 内 n/i 的值相同
}
```

2. 线性代数

2.1 高斯消元

```
int Gauss(vector<vector<double>> a, vector<double> &ans){
    int n = a.size(), m = a[0].size() - 1, r = 0;
    for(int c = 0; c < m && r < n; ++c){
        int p = r;
        for(int i = r; i < n; ++i) if(fabs(a[i][c]) > fabs(a[p][c])) p = i;
        if(fabs(a[p][c]) < 1e-9) continue;
        swap(a[p], a[r]);
        double div = a[r][c];
        for(int j = c; j <= m; ++j) a[r][j] /= div;
        for(int i = 0; i < n; ++i) if(i != r && fabs(a[i][c]) > 1e-9){
            double t = a[i][c];
            for(int j = c; j <= m; ++j) a[i][j] -= t * a[r][j];
        }
        ++r;
    }
```

```

    }
    ans.assign(m,0);
    for(int i=0;i<r;++i){
        int pos=0;
        while(pos<m&&fabs(a[i][pos])<1e-9) ++pos;
        if(pos<m) ans[pos]=a[i][m];
    }
    return r;
}

```

2.2 矩阵快速幂

```

struct Mat{
    int n;
    vector<vector<ll>>> a;
    Mat(int _n=0,int id=0):n(_n),a(_n,vector<ll>(_n,0)){
        if(id) for(int i=0;i<n;++i) a[i][i]=1;
    }
};

Mat operator*(const Mat &A,const Mat &B){
    Mat C(A.n);
    for(int i=0;i<A.n;++i)
        for(int k=0;k<A.n;++k) if(A.a[i][k])
            for(int j=0;j<A.n;++j)
                C.a[i][j]+=A.a[i][k]*B.a[k][j];
    return C;
}

```

2.3 线性基

```

ll basis[61];
void insert(ll x){
    for(int i=60;i>=0;--i){
        if(!(x>>i&1)) continue;
        if(!basis[i]) return basis[i]=x,void();
        x^=basis[i];
    }
}

ll queryMax(ll x){
    for(int i=60;i>=0;--i) x=max(x,x^basis[i]);
    return x;
}

```

2.4 行列式

```

long double det(vector<vector<long double>>> a){
    int n=a.size();
    long double ans=1;
    for(int i=0;i<n;++i){
        int p=i;
        for(int j=i;j<n;++j) if(fabsl(a[j][i])>fabsl(a[p][i])) p=j;
        if(fabsl(a[p][i])<1e-18) return 0;
        if(i!=p){ swap(a[i],a[p]); ans=-ans; }
        ans*=a[i][i];
    }
}

```

```

        for(int j=i+1;j<n;++j){
            long double t=a[j][i]/a[i][i];
            for(int k=i;k<n;++k) a[j][k]-=t*a[i][k];
        }
    }
    return ans;
}

```

2.5 矩阵求逆

```

bool inverse(vector<vector<double>> a,vector<vector<double>> &inv){
    int n=a.size();
    inv.assign(n,vector<double>(n,0));
    for(int i=0;i<n;++i) inv[i][i]=1;
    for(int c=0;c<n;++c){
        int p=c;
        for(int i=c;i<n;++i) if(fabs(a[i][c])>fabs(a[p][c])) p=i;
        if(fabs(a[p][c])<1e-9) return 0;
        swap(a[p],a[c]);
        swap(inv[p],inv[c]);
        double div=a[c][c];
        for(int j=0;j<n;++j) a[c][j]/=div,inv[c][j]/=div;
        for(int i=0;i<n;++i) if(i!=c){
            double t=a[i][c];
            for(int j=0;j<n;++j) a[i][j]-=t*a[c][j],inv[i][j]-=t*inv[c][j];
        }
    }
    return 1;
}

```

2.6 递推转矩阵

线性递推 $f_n = c_1 f_{n-1} + \dots + c_k f_{n-k}$ 可写成矩阵快速幂。

```

// 形如 f[n] = sum c[i]*f[n-i]
// 构造 k*k 转移矩阵后快速幂

```

3. 组合数学

3.1 组合数预处理

```

ll C[2005][2005];
void initC(int n){
    for(int i=0;i<=n;++i){
        C[i][0]=C[i][i]=1;
        for(int j=1;j<i;++j)
            C[i][j]=(C[i-1][j-1]+C[i-1][j])%p;
    }
}

```

3.2 组合数取模（阶乘）

```

11 fac[N],ifac[N];
void initFac(int n){
    fac[0]=1;
    for(int i=1;i<=n;++i) fac[i]=fac[i-1]*i%p;
    ifac[n]=qpow(fac[n],p-2,p);
    for(int i=n;i>=1;--i) ifac[i-1]=ifac[i]*i%p;
}

11 C(ll n,ll m){
    if(n<m||m<0) return 0;
    return fac[n]*ifac[m]%p*ifac[n-m]%p;
}

```

3.3 容斥原理

$$\left| \bigcup_i A_i \right| = \sum |A_i| - \sum |A_i \cap A_j| + \sum |A_i \cap A_j \cap A_k| - \dots$$

常用于“至少一个满足”“没有任何满足”的计数问题。

3.4 Catalan 数

$$Cat_n = \frac{1}{n+1} \binom{2n}{n}$$

```

11 Catalan(int n){
    return C(2*n,n)-C(2*n,n-1);
}

```

3.5 排列与错排

```

11 der[N];
void initDer(int n){
    der[0]=1; der[1]=0;
    for(int i=2;i<=n;++i) der[i]=(i-1)*(der[i-1]+der[i-2])%p;
}

```

错排公式：

$$!n = n! \sum_{i=0}^n \frac{(-1)^i}{i!}$$

3.6 Stirling 数

第二类 Stirling 数 $S(n, k)$ 表示把 n 个不同元素划分为 k 个非空集合的方案数。

$$S(n, k) = S(n-1, k-1) + kS(n-1, k)$$

```

11 S[205][205];
void initS(int n){
    S[0][0]=1;
    for(int i=1;i<=n;++i)
        for(int j=1;j<=i;++j)
            S[i][j]=(S[i-1][j-1]+j*S[i-1][j])%p;
}

```

3.7 Bell 数

Bell 数表示集合划分总数：

$$B_n = \sum_{k=0}^n S(n, k)$$

3.8 多重集与插板法

将 n 个相同物品分到 k 个盒子：

$$\binom{n+k-1}{k-1}$$

若每盒至少 1 个：

$$\binom{n-1}{k-1}$$

3.9 Burnside 引理

设群 G 作用在集合 X 上，则不同本质状态数为

$$\frac{1}{|G|} \sum_{g \in G} |\text{Fix}(g)|$$

3.10 Pólya 定理

用于“旋转/翻转等价”的颜色计数，可配合循环指标生成函数使用。

4. 多项式与生成函数

4.1 NTT

```

void ntt(vector<ll> &a,int inv){
    int n=a.size();
    // 先做 bit-reverse 置换
    for(int len=1;len<n;len<<=1){
        ll wn=qpow(G,(p-1)/(len<<1),p);
        if(inv==-1) wn=qpow(wn,p-2,p);
        for(int i=0;i<n;i+=len<<1){
            ll w=1;

```

```

        for(int j=0;j<len;++j){
            ll x=a[i+j],y=w*a[i+j+len]%p;
            a[i+j]=(x+y)%p;
            a[i+j+len]=(x-y+p)%p;
            w=w*wn%p;
        }
    }
}
if(inv==-1){
    ll invn=qpow(n,p-2,p);
    for(auto &x:a) x=x*invn%p;
}
}

```

4.1.1 NTT 半家桶 (ln/exp/pow/sqrt)

```

// 1) 模数 p 为 NTT 友好质数 (如 998244353), 原根 G
// 2) qpow(a,b,p)
// 3) ntt(vector<ll>& a, int inv) (inv=1 正变换, -1 逆变换)

vector<ll> polyCut(vector<ll> a,int n){
    if((int)a.size()>n) a.resize(n);
    return a;
}

vector<ll> polyDeriv(const vector<ll> &a){
    if(a.empty()) return {};
    vector<ll> b(max(0,(int)a.size()-1));
    for(int i=1;i<(int)a.size();++i) b[i-1]=a[i]*i%p;
    return b;
}

vector<ll> polyInteg(const vector<ll> &a){
    vector<ll> b((int)a.size()+1);
    for(int i=0;i<(int)a.size();++i) b[i+1]=a[i]*qpow(i+1,p-2,p)%p;
    return b;
}

vector<ll> polyMul(vector<ll> a,vector<ll> b){
    if(a.empty()||b.empty()) return {};
    int n=(int)a.size(),m=(int)b.size(),lim=1;
    while(lim<n+m-1) lim<=<=1;
    a.resize(lim); b.resize(lim);
    ntt(a,1); ntt(b,1);
    for(int i=0;i<lim;++i) a[i]=a[i]*b[i]%p;
    ntt(a,-1);
    a.resize(n+m-1);
    return a;
}

// A[0] != 0
vector<ll> polyInv(const vector<ll> &A,int n){
    vector<ll> B(1,qpow(A[0],p-2,p));
    for(int len=2;len<=n;len<=<=1){
        vector<ll> F(min((int)A.size(),len));
        for(int i=0;i<(int)F.size();++i) F[i]=A[i];
        vector<ll> T=polyMul(polyMul(B,B),F);
        B.resize(len);
        for(int i=0;i<len;++i){
            ll v=(i<(int)T.size()?T[i]:0);

```

```

        B[i]=(2LL*B[i]%p-v+p)%p;
    }
}
B.resize(n);
return B;
}

// ln(A) = integral(A' / A), 要求 A[0] = 1
vector<ll> polyLn(const vector<ll> &A,int n){
    vector<ll> d=polyDeriv(A);
    vector<ll> invA=polyInv(A,n);
    vector<ll> q=polyMul(d,invA);
    q=polyCut(q,n-1);
    vector<ll> res=polyInteg(q);
    return polyCut(res,n);
}

// exp(A), 要求 A[0] = 0
vector<ll> polyExp(const vector<ll> &A,int n){
    vector<ll> B(1,1); // exp(0)=1
    for(int len=2;len<=n;len<=1){
        vector<ll> lnB=polyLn(B,len);
        vector<ll> C(len);
        for(int i=0;i<len;++i){
            ll ai=(i<(int)A.size()?A[i]:0);
            ll li=(i<(int)lnB.size()?lnB[i]:0);
            C[i]=(ai-li+p)%p;
        }
        C[0]=(C[0]+1)%p;
        B=polyMul(B,C);
        B.resize(len);
    }
    B.resize(n);
    return B;
}

// sqrt(A), 要求给定  $s_0^2 \equiv A[0] \pmod p$ , 且  $A[0] \neq 0$ 
vector<ll> polySqrt(const vector<ll> &A,int n,ll s0){
    ll inv2=(p+1)/2;
    vector<ll> B(1,s0%p);
    for(int len=2;len<=n;len<=1){
        vector<ll> invB=polyInv(B,len);
        vector<ll> F(min((int)A.size(),len));
        for(int i=0;i<(int)F.size();++i) F[i]=A[i];
        vector<ll> T=polyMul(F,invB);
        B.resize(len);
        for(int i=0;i<len;++i){
            ll ti=(i<(int)T.size()?T[i]:0);
            B[i]=(B[i]+ti)%p*inv2%p;
        }
    }
    B.resize(n);
    return B;
}

// pow(A,k): 返回  $A^k \bmod x^n$ 
// 要点: 处理前导零。若  $A = x^t * B$ , 则  $A^k = x^{(t*k)} * B^k$ 
vector<ll> polyPow(vector<ll> A,ll k,int n){
    if(n==0) return {};
    int t=0;
    while(t<(int)A.size()&&A[t]==0) ++t;
    if(t==(int)A.size()) return vector<ll>(n,0);
    if(1LL*t*k>=n) return vector<ll>(n,0);

```

```

    ll lead=A[t];
    ll invLead=qpow(lead,p-2,p);
    vector<ll> B(A.begin()+t,A.end());
    for(auto &x:B) x=x*invLead%p;

    int m=n-(int)(1LL*t*k);
    B=polyCut(B,m);

    // B[0] = 1
    vector<ll> lnB=polyLn(B,m);
    for(auto &x:lnB) x=x*(k%(p-1))%p;
    vector<ll> C=polyExp(lnB,m);

    ll coef=qpow(lead,k%(p-1),p);
    for(auto &x:C) x=x*coef%p;

    vector<ll> res(n,0);
    int shift=(int)(1LL*t*k);
    for(int i=0;i<(int)C.size();&&i+shift<n;++i) res[i+shift]=C[i];
    return res;
}

```

4.2 多项式求逆

设 $B(x) = A(x)^{-1}$, 则

$$B(x) \cdot A(x) \equiv 1 \pmod{x^n}$$

常用递推: $B_{k+1} = B_k(2 - AB_k) \bmod x^{2^k}$ 。

4.3 多项式求导与积分

```

vector<ll> deriv(vector<ll> a){
    vector<ll> b(max(0,(int)a.size()-1));
    for(int i=1;i<(int)a.size();++i) b[i-1]=a[i]*i%p;
    return b;
}

vector<ll> integ(vector<ll> a){
    vector<ll> b(a.size()+1);
    for(int i=0;i<(int)a.size();++i) b[i+1]=a[i]*qpow(i+1,p-2,p)%p;
    return b;
}

```

4.4 生成函数常见结论

- 普通生成函数: $F(x) = \sum a_n x^n$
- 指数生成函数: $F(x) = \sum a_n \frac{x^n}{n!}$
- 卷积对应乘法: 若 $c_n = \sum_{i+j=n} a_i b_j$, 则 $C(x) = A(x)B(x)$

4.5 FWT（按位卷积）

4.5.1 异或卷积

```
void fwt_xor(vector<ll> &a,int inv){
    int n=a.size();
    for(int len=1;len<n;len<=1)
        for(int i=0;i<n;i+=len<<1)
            for(int j=0;j<len;++j){
                ll x=a[i+j],y=a[i+j+len];
                a[i+j]=(x+y)%p;
                a[i+j+len]=(x-y+p)%p;
            }
    if(inv==1){
        ll invn=qpow(n,p-2,p);
        for(auto &x:a) x=x*invn%p;
    }
}
```

4.5.2 或卷积

```
void fwt_or(vector<ll> &a,int inv){
    int n=a.size();
    for(int len=1;len<n;len<=1)
        for(int i=0;i<n;i+=len<<1)
            for(int j=0;j<len;++j)
                if(inv==1) a[i+j+len]=(a[i+j+len]+a[i+j])%p;
                else a[i+j+len]=(a[i+j+len]-a[i+j]+p)%p;
}
```

4.5.3 与卷积

```
void fwt_and(vector<ll> &a,int inv){
    int n=a.size();
    for(int len=1;len<n;len<=1)
        for(int i=0;i<n;i+=len<<1)
            for(int j=0;j<len;++j)
                if(inv==1) a[i+j]=(a[i+j]+a[i+j+len])%p;
                else a[i+j]=(a[i+j]-a[i+j+len]+p)%p;
}
```

4.6 拉格朗日插值

已知 $n + 1$ 个点 (x_i, y_i) ，求多项式在某点 k 的值。

$$f(k) = \sum_{i=0}^n y_i \prod_{j \neq i} \frac{k - x_j}{x_i - x_j}$$

```
ll Lagrange(vector<ll> x,vector<ll> y,ll k){
    int n=x.size()-1;
    ll ans=0;
    for(int i=0;i<=n;++i){
```

```

    ll num=1,den=1;
    for(int j=0;j<=n;++j) if(i!=j){
        num=num*(k-x[j])%p;
        den=den*(x[i]-x[j])%p;
    }
    ans=(ans+y[i]*num%p*qpow((den%p+p)%p,p-2,p))%p;
}
return (ans%p+p)%p;
}

```

4.7 Newton 迭代（多项式）

常用于求根、求开方、求对数、求指数。

$$g_{k+1} = g_k - \frac{f(g_k)}{f'(g_k)}$$

4.8 生成函数常用模型

- 斐波那契: $\frac{x}{1-x-x^2}$
- 等差级数: $\frac{x}{(1-x)^2}$
- 幂和: $\sum_{n \geq 0} n^k x^n$ 可由算子法求出
- 组合卷积: $\frac{1}{1-A(x)} = 1 + A(x) + A(x)^2 + \dots$

5. 数值算法

5.1 三分法

```

double ternary(double l,double r){
    for(int it=0;it<200;++it){
        double m1=l+(r-l)/3,m2=r-(r-l)/3;
        if(f(m1)<f(m2)) l=m1;
        else r=m2;
    }
    return (l+r)/2;
}

```

5.2 Simpson 积分

```

double simpson(double l,double r){
    double m=(l+r)/2;
    return (f(l)+4*f(m)+f(r))*(r-l)/6;
}

double asr(double l,double r,double eps,double whole){
    double m=(l+r)/2;
    double left=simpson(l,m),right=simpson(m,r);
    if(fabs(left+right-whole)<=15*eps) return left+right+(left+right-whole)/15;
    return asr(l,m,eps/2,left)+asr(m,r,eps/2,right);
}

```

5.3 牛顿迭代

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

5.4 二分答案

若答案满足单调性，则可二分枚举。

```
bool check(double mid){
    return /* 单调判定 */;
}

double bs(double l,double r){
    for(int it=0;it<100;++it){
        double m=(l+r)/2;
        if(check(m)) r=m;
        else l=m;
    }
    return r;
}
```

5.5 牛顿迭代求平方根

$$x_{k+1} = \frac{x_k + \frac{a}{x_k}}{2}$$

5.6 黄金分割

```
double golden(double l,double r){
    const double phi=(sqrt(5.0)-1)/2;
    double x1=r-phi*(r-l),x2=l+phi*(r-l);
    for(int it=0;it<200;++it){
        if(f(x1)<f(x2)) r=x2,x2=x1,x1=r-phi*(r-l);
        else l=x1,x1=x2,x2=l+phi*(r-l);
    }
    return (l+r)/2;
}
```

6. 博弈论

6.1 Nim 游戏

```
int sg=0;
for(int x:a) sg^=x;
cout<<(sg?"Alice":"Bob")<<'\n';
```

6.2 SG 函数

```
int dfs(int u){
    if(~sg[u]) return sg[u];
    set<int> s;
    for(int v:g[u]) s.insert(dfs(v));
    int mex=0;
    while(s.count(mex)) ++mex;
    return sg[u]=mex;
}
```

6.3 DAG 博弈

若状态转移图是 DAG，则可以按拓扑序/记忆化搜索求 SG 值。

```
vector<int> g[N];
int sg[N];

int dfs(int u){
    if(~sg[u]) return sg[u];
    set<int> s;
    for(int v:g[u]) s.insert(dfs(v));
    int mex=0;
    while(s.count(mex)) ++mex;
    return sg[u]=mex;
}
```

6.4 威佐夫博弈

```
#include<bits/stdc++.h>
using namespace std;
const double l=(sqrt(5.0)+1.0)/2.0;
int n,m;
int main(){
    scanf("%d %d",&n,&m);
    if(n<m) swap(n,m);
    int d=n-m;
    if(int((double)d*l)==m) puts("0");
    else puts("1");
    return 0;
}
```

6.5 斐波那契尼姆博弈

```
#include<bits/stdc++.h>
long long n,x,y,z;
int main(){
    scanf("%lld",&n);
    while(66){
        if(n==1) return 0&puts("1");
        if(n==2) return 0&puts("2");
        x=1,y=2,z=3;
        while(z<n) x=y,y=z,z=x+y;
        if(z==n) return 0&printf("%lld",z);
    }
}
```

```

        else n-=y;
    }
    return 0;
}

```

6.6 mex 性质

$$SG(u) = \text{mex}\{SG(v) \mid u \rightarrow v\}$$

其中 mex 表示最小非负整数。

6.7 记忆化模板

```

int win[N];
int solve(int u){
    if(win[u]!=-1) return win[u];
    for(int v:g[u])
        if(!solve(v)) return win[u]=1;
    return win[u]=0;
}

```

7. 计算几何

7.1 计算几何基础模板

```

// 1) 默认使用 long double, 提升精度稳定性。
// 2) 所有判等都通过 dcmp 与 eps 完成, 避免直接比较浮点。
#include <bits/stdc++.h>
using namespace std;

using ld = long double;
const ld EPS = 1e-12;

int dcmp(ld x){
    if(fabs(x) < EPS) return 0;
    return x < 0 ? -1 : 1;
}

struct Point{
    ld x,y;
    Point(ld _x=0,ld _y=0): x(_x),y(_y) {}

    Point operator+(const Point& o) const { return Point(x+o.x,y+o.y); }
    Point operator-(const Point& o) const { return Point(x-o.x,y-o.y); }
    Point operator*(ld k) const { return Point(x*k,y*k); }
    Point operator / (ld k) const { return Point(x/k,y/k); }

    bool operator == (const Point& o) const {
        return dcmp(x-o.x)==0 && dcmp(y-o.y)==0;
    }

    bool operator < (const Point& o) const {
        if(dcmp(x-o.x)!=0) return x<o.x;
        return y<o.y;
    }
}

```

```

    }
};

using Vec = Point;

// 叉积  $a \times b$ 
ld cross(const Vec& a, const Vec& b){ return a.x*b.y-a.y*b.x; }
// 点积  $a \cdot b$ 
ld dot(const Vec& a, const Vec& b){ return a.x*b.x+a.y*b.y; }
// 向量长度
ld norm(const Vec& a){ return sqrtl(dot(a,a)); }

// 方向判断:  $a \rightarrow b$  与  $a \rightarrow c$  的转向
ld orient(const Point& a, const Point& b, const Point& c){
    return cross(b-a, c-a);
}

// 点 p 是否在线段 [a,b] 上 (含端点)
bool onSegment(const Point& p, const Point& a, const Point& b){
    return dcmp(cross(p-a, p-b))==0 && dcmp(dot(p-a, p-b))<=0;
}

// 线段 [a,b] 与 [c,d] 是否相交 (含端点/重叠)
bool segIntersect(const Point& a, const Point& b, const Point& c, const Point& d){
    ld c1 = orient(a,b,c), c2 = orient(a,b,d);
    ld c3 = orient(c,d,a), c4 = orient(c,d,b);

    if(dcmp(c1)*dcmp(c2)<0 && dcmp(c3)*dcmp(c4)<0) return true; // 严格相交
    if(onSegment(c,a,b) || onSegment(d,a,b) || onSegment(a,c,d) || onSegment(b,c,d)) return
        ↪ true;
    return false;
}

// 直线 AB 与 CD 的交点 (需先保证不平行)
bool lineIntersection(const Point& a, const Point& b, const Point& c, const Point& d, Point&
    ↪ out){
    Vec u = b-a, v = d-c;
    ld den = cross(u,v);
    if(dcmp(den)==0) return false; // 平行或重合
    ld t = cross(c-a, v) / den;
    out = a+u*t;
    return true;
}

// 点到直线 AB 的距离
ld distPointLine(const Point& p, const Point& a, const Point& b){
    if(a==b) return norm(p-a);
    return fabs1(cross(b-a, p-a)) / norm(b-a);
}

// 点到线段 AB 的距离
ld distPointSegment(const Point& p, const Point& a, const Point& b){
    if(a==b) return norm(p-a);
    if(dcmp(dot(p-a, b-a))<0) return norm(p-a);
    if(dcmp(dot(p-b, a-b))<0) return norm(p-b);
    return distPointLine(p, a, b);
}

// 点 p 在直线 AB 上的投影点
Point projection(const Point& p, const Point& a, const Point& b){
    Vec v = b-a;
    if(a==b) return a;
    ld t = dot(p-a, v) / dot(v,v);

```

```

    return a+v*t;
}

// 点 p 关于直线 AB 的对称点
Point reflection(const Point& p,const Point& a,const Point& b){
    Point q = projection(p,a,b);
    return q*2-p;
}

// 多边形有向面积（逆时针为正）
ld polygonArea2(const vector<Point>& p){
    int n = (int)p.size();
    ld s = 0;
    for(int i=0;i<n;++i){
        s += cross(p[i],p[(i+1)%n]);
    }
    return s; // = 2*area
}

ld polygonArea(const vector<Point>& p){
    return fabs1(polygonArea2(p)) / 2.0;
}

// 点在多边形: 0= 外部,1= 内部,2= 边界（多边形可为非凸）
int pointInPolygon(const vector<Point>& poly,const Point& a){
    int n = (int)poly.size();
    bool in = false;
    for(int i=0,j=n-1;i<n;j=i++){
        const Point &p=poly[i],&q=poly[j];
        if(onSegment(a,p,q)) return 2;
        bool inter = ((dcmp(p.y-a.y)>0)!= (dcmp(q.y-a.y)>0)) &&
            (dcmp(a.x-(q.x-p.x)*(a.y-p.y)/(q.y-p.y)-p.x) < 0);
        if(inter) in = !in;
    }
    return in ? 1 : 0;
}

// Andrew 凸包（返回逆时针，不重复首尾点）
vector<Point> convexHull(vector<Point> p){
    sort(p.begin(),p.end());
    p.erase(unique(p.begin(),p.end(),[](const Point& a,const Point& b){return
        ↪ a==b;}),p.end());
    int n = (int)p.size();
    if(n<=1) return p;

    vector<Point> st;
    // 下凸壳
    for(int i=0;i<n;++i){
        while((int)st.size()>=2 && dcmp(orient(st[(int)st.size()-2],st.back(),p[i]))<=0)
            ↪ st.pop_back();
        st.push_back(p[i]);
    }
    // 上凸壳
    int t = (int)st.size();
    for(int i=n-2;i>=0;--i){
        while((int)st.size()>t && dcmp(orient(st[(int)st.size()-2],st.back(),p[i]))<=0)
            ↪ st.pop_back();
        st.push_back(p[i]);
    }
    st.pop_back();
    return st;
}

```

7.2 凸包相关

7.2.1 旋转卡壳求凸包直径

```
// 输入为逆时针凸包 ch (无重复首尾点)
// 返回最远点对的平方距离
long double convexDiameter2(const vector<Point>& ch){
    int n=(int)ch.size();
    if(n==1) return 0;
    if(n==2) return dot(ch[0]-ch[1],ch[0]-ch[1]);
    int j=1;
    long double ans=0;
    for(int i=0;i<n;++i){
        int ni=(i+1)%n;
        while(fabs1(cross(ch[ni]-ch[i],ch[(j+1)%n]-ch[i])) >
            ↪ fabs1(cross(ch[ni]-ch[i],ch[j]-ch[i])))
            j=(j+1)%n;
        ans=max(ans,dot(ch[i]-ch[j],ch[i]-ch[j]));
        ans=max(ans,dot(ch[ni]-ch[j],ch[ni]-ch[j]));
    }
    return ans;
}
```

7.2.2 Minkowski 和 (凸多边形)

```
// 说明: 输入 A/B 必须是逆时针凸包, 且首元素为字典序最小点。
// 输出 C = A (+) B 的逆时针凸包 (不重复首尾)。
vector<Point> minkowskiSum(vector<Point> A,vector<Point> B){
    auto shiftMin = [&](vector<Point>& P){
        int n=P.size(),id=0;
        for(int i=1;i<n;++i) if(P[i]<P[id]) id=i;
        rotate(P.begin(),P.begin()+id,P.end());
    };
    shiftMin(A); shiftMin(B);
    int n=A.size(),m=B.size();
    vector<Vec> EA,EB;
    for(int i=0;i<n;++i) EA.push_back(A[(i+1)%n]-A[i]);
    for(int i=0;i<m;++i) EB.push_back(B[(i+1)%m]-B[i]);

    vector<Point> C;
    C.push_back(A[0]+B[0]);
    int i=0,j=0;
    while(i<n || j<m){
        Vec va = (i<n?EA[i]:Vec(0,0));
        Vec vb = (j<m?EB[j]:Vec(0,0));
        if(i<n && (j==m || dcmp(cross(va,vb))>0)) C.push_back(C.back()+va),++i;
        else if(j<m && (i==n || dcmp(cross(va,vb))<0)) C.push_back(C.back()+vb),++j;
        else C.push_back(C.back()+va+vb),++i,++j;
    }
    C.pop_back();
    return convexHull(C); // 去共线冗余点
}
```

7.3 圆与直线

```
struct Circle{
    Point o;
```

```

    ld r;
    Circle(Point _o=Point(),ld _r=0): o(_o),r(_r) {}
};

// 直线 AB 与圆 C 的交点个数 (0/1/2), 交点存入 out
int lineCircleIntersection(const Point& a,const Point& b,const Circle& C,vector<Point>& out){
    out.clear();
    Point h = projection(C.o,a,b);          // 圆心到直线投影
    ld d = norm(h-C.o);
    if(dcmp(d-C.r) > 0) return 0;
    Vec v = b-a;
    ld len = norm(v);
    if(dcmp(len)==0) return 0;
    v = v / len;
    if(dcmp(d-C.r)==0){
        out.push_back(h);
        return 1;
    }
    ld t = sqrtl(max((ld)0,C.r*C.r-d*d));
    out.push_back(h+v*t);
    out.push_back(h-v*t);
    return 2;
}

// 两圆位置关系:
// 0: 外离,1: 外切,2: 相交,3: 内切,4: 内含
int circleRelation(const Circle& A,const Circle& B){
    ld d = norm(A.o-B.o);
    if(dcmp(d-(A.r+B.r))>0) return 0;
    if(dcmp(d-(A.r+B.r))==0) return 1;
    if(dcmp(d-fabsl(A.r-B.r))>0) return 2;
    if(dcmp(d-fabsl(A.r-B.r))==0) return 3;
    return 4;
}

```

7.4 最近点对 (分治)

```

// 最近点对 O(n log n) (返回最小距离平方)
long double closestPairRec(vector<Point>& p,int l,int r,vector<Point>& tmp){
    if(r-l<=3){
        long double ans=1e100;
        for(int i=l;i<=r;++i)
            for(int j=i+1;j<=r;++j)
                ans=min(ans,dot(p[i]-p[j],p[i]-p[j]));
        sort(p.begin()+l,p.begin()+r+1,[](const Point& a,const Point& b){return a.y<b.y;});
        return ans;
    }
    int m=(l+r)>>1;
    ld midx=p[m].x;
    ld d=min(closestPairRec(p,l,m,tmp),closestPairRec(p,m+1,r,tmp));
    merge(p.begin()+l,p.begin()+m+1,p.begin()+m+1,p.begin()+r+1,tmp.begin(),
        [](const Point& a,const Point& b){return a.y<b.y;});
    copy(tmp.begin(),tmp.begin()+(r-l+1),p.begin()+l);

    int ts=0;
    for(int i=l;i<=r;++i) if((p[i].x-midx)*(p[i].x-midx) < d) tmp[ts++]=p[i];
    for(int i=0;i<ts;++i)
        for(int j=i+1;j<ts && (tmp[j].y-tmp[i].y)*(tmp[j].y-tmp[i].y)<d;++j)
            d=min(d,dot(tmp[i]-tmp[j],tmp[i]-tmp[j]));
    return d;
}

```

```

}

long double closestPair2(vector<Point> p){
    sort(p.begin(),p.end(),[](const Point& a,const Point& b){
        if(dcmp(a.x-b.x)!=0) return a.x<b.x;
        return a.y<b.y;
    });
    vector<Point> tmp(p.size());
    return closestPairRec(p,0,(int)p.size()-1,tmp);
}

```

8. 杂项

8.1 高精度全家桶

```

// OI-wiki
struct BigIntTiny {
    int sign;
    std::vector<int> v;

    BigIntTiny() : sign(1) {}
    BigIntTiny(const std::string &s) { *this = s; }
    BigIntTiny(int v) {
        char buf[21];
        sprintf(buf,"%d",v);
        *this = buf;
    }
    void zip(int unzip) {
        if (unzip == 0) {
            for (int i = 0; i < (int)v.size(); i++)
                v[i] = get_pos(i*4)+get_pos(i*4+1)*10+get_pos(i*4+2)*100+get_pos(i*4+3)*1000;
        } else
            for (int i = (v.resize(v.size()*4),(int)v.size()-1),a; i >= 0; i--)
                a = (i % 4 >= 2) ? v[i / 4] / 100 : v[i / 4] % 100,v[i] = (i & 1) ? a / 10 :
                    ↪ a % 10;
        setsign(1,1);
    }
    int get_pos(unsigned pos) const { return pos >= v.size() ? 0 : v[pos]; }
    BigIntTiny &setsign(int newsign,int rev) {
        for (int i = (int)v.size()-1; i > 0 && v[i] == 0; i--)
            v.erase(v.begin()+i);
        sign = (v.size() == 0 || (v.size() == 1 && v[0] == 0)) ? 1 : (rev ? newsign*sign :
            ↪ newsign);
        return *this;
    }
    std::string to_str() const {
        BigIntTiny b = *this;
        std::string s;
        for (int i = (b.zip(1),0); i < (int)b.v.size(); ++i)
            s += char(*(b.v.rbegin()+i)+'0');
        return (sign < 0 ? "-" : "")+(s.empty() ? std::string("0") : s);
    }
    bool absless(const BigIntTiny &b) const {
        if (v.size() != b.v.size()) return v.size() < b.v.size();
        for (int i = (int)v.size()-1; i >= 0; i--)
            if (v[i] != b.v[i]) return v[i] < b.v[i];
        return false;
    }
    BigIntTiny operator-() const {

```

```

    BigIntTiny c = *this;
    c.sign = (v.size() > 1 || v[0]) ? -c.sign : 1;
    return c;
}
BigIntTiny &operator=(const std::string &s) {
    if (s[0] == '-')
        *this = s.substr(1);
    else {
        for (int i = (v.clear(),0); i < (int)s.size(); ++i)
            v.push_back(*(s.rbegin()+i)-'0');
        zip(0);
    }
    return setsign(s[0] == '-' ? -1 : 1, sign = 1);
}
bool operator<(const BigIntTiny &b) const {
    return sign != b.sign ? sign < b.sign : (sign == 1 ? absless(b) : b.absless(*this));
}
bool operator==(const BigIntTiny &b) const { return v == b.v && sign == b.sign; }
BigIntTiny &operator+=(const BigIntTiny &b) {
    if (sign != b.sign) return *this = (*this)--b;
    v.resize(std::max(v.size(), b.v.size())+1);
    for (int i = 0, carry = 0; i < (int)b.v.size() || carry; i++) {
        carry += v[i]+b.get_pos(i);
        v[i] = carry % 10000, carry /= 10000;
    }
    return setsign(sign,0);
}
BigIntTiny operator+(const BigIntTiny &b) const {
    BigIntTiny c = *this;
    return c += b;
}
void add_mul(const BigIntTiny &b, int mul) {
    v.resize(std::max(v.size(), b.v.size())+2);
    for (int i = 0, carry = 0; i < (int)b.v.size() || carry; i++) {
        carry += v[i]+b.get_pos(i)*mul;
        v[i] = carry % 10000, carry /= 10000;
    }
}
BigIntTiny operator-(const BigIntTiny &b) const {
    if (b.v.empty() || b.v.size() == 1 && b.v[0] == 0) return *this;
    if (sign != b.sign) return (*this)+b;
    if (absless(b)) return -(b-*this);
    BigIntTiny c;
    for (int i = 0, borrow = 0; i < (int)v.size(); i++) {
        borrow += v[i]-b.get_pos(i);
        c.v.push_back(borrow);
        c.v.back() -= 10000*(borrow >= 31);
    }
    return c.setsign(sign,0);
}
BigIntTiny operator*(const BigIntTiny &b) const {
    if (b < *this) return b**this;
    BigIntTiny c,d = b;
    for (int i = 0; i < (int)v.size(); i++, d.v.insert(d.v.begin(),0))
        c.add_mul(d,v[i]);
    return c.setsign(sign*b.sign,0);
}
BigIntTiny operator/(const BigIntTiny &b) const {
    BigIntTiny c,d;
    BigIntTiny e=b;
    e.sign=1;

    d.v.resize(v.size());

```

```

double db = 1.0 / (b.v.back()+(b.get_pos((unsigned)b.v.size()-2) / 1e4) +
                  (b.get_pos((unsigned)b.v.size()-3)+1) / 1e8);
for (int i = (int)v.size()-1; i >= 0; i--) {
    c.v.insert(c.v.begin(),v[i]);
    int m =
        ↪ (int)((c.get_pos((int)e.v.size())*10000+c.get_pos((int)e.v.size()-1))*db);
    c = c-e*m,c.setsign(c.sign,0),d.v[i] += m;
    while (!(c < e))
        c = c-e,d.v[i] += 1;
}
return d.setsign(sign*b.sign,0);
}
BigIntTiny operator%(const BigIntTiny &b) const { return *this-*this / b*b; }
bool operator>(const BigIntTiny &b) const { return b < *this; }
bool operator<=(const BigIntTiny &b) const { return !(b < *this); }
bool operator>=(const BigIntTiny &b) const { return !(*this < b); }
bool operator!=(const BigIntTiny &b) const { return !(*this == b); }
};

```

8.2 快速平方根取倒数

```

loat Q_rsqrt( float number ){
    long i;
    float x2,y;
    const float threehalfs = 1.5F;
    x2 = number*0.5F;
    y = number;
    i  =*( long*) &y;
    y  =*( float*) &i;
    return y;
}

```

9. 数据结构

9.1 FHQ Treap

```

struct FHQ {
    struct Node {
        int ls, rs, val, siz, rnd;
    }t[MAXN];
    int rt, cnt;
    void pushup(int p) {
        t[p].siz = t[t[p].ls].siz + t[t[p].rs].siz + 1;
    }
    int New(int a) {
        t[++cnt].val = a;
        t[cnt].siz = 1;
        t[cnt].rnd = Rand();
        return cnt;
    }
    void split(int now, int k, int &x, int &y) {
        if (!now) {
            x = y = 0;
            return;
        }
        if (t[now].val <= k) {

```

```

        x = now;
        split(t[now].rs, k, t[now].rs, y);
    }
    else {
        y = now;
        split(t[now].ls, k, x, t[now].ls);
    }
    pushup(now);
}
int merge(int u, int v) {
    if (!u || !v) return u | v;
    if (t[u].rnd < t[v].rnd) {
        t[u].rs = merge(t[u].rs, v);
        pushup(u);
        return u;
    }
    t[v].ls = merge(u, t[v].ls);
    pushup(v);
    return v;
}
void insert(int a) {
    int x, y;
    split(rt, a - 1, x, y);
    rt = merge(merge(x, New(a)), y);
}
void erase(int a) {
    int x, y, z;
    split(rt, a - 1, x, y);
    split(y, a, y, z);
    y = merge(t[y].ls, t[y].rs);
    rt = merge(merge(x, y), z);
}
int rk(int a) {
    int x, y;
    split(rt, a - 1, x, y);
    int ret = t[x].siz + 1;
    rt = merge(x, y);
    return ret;
}
int kth(int now, int k) {
    if (k <= t[t[now].ls].siz) return kth(t[now].ls, k);
    if (t[t[now].ls].siz + 1 < k) return kth(t[now].rs, k - t[t[now].ls].siz - 1);
    return t[now].val;
}
int pre(int a) {
    int x, y;
    split(rt, a - 1, x, y);
    int ret = kth(x, t[x].siz);
    rt = merge(x, y);
    return ret;
}
int nxt(int a) {
    int x, y;
    split(rt, a, x, y);
    int ret = kth(y, 1);
    rt = merge(x, y);
    return ret;
}
}t;

```

9.2 CDQ 分治

```

auto cdq = [&](int l, int r, auto &self) -> void {
    if (l == r) return;
    int mid = (l + r) >> 1;
    self(l, mid, self); self(mid + 1, r, self);
    int p1 = l, p2 = mid + 1, cur = l;
    while (p1 <= mid && p2 <= r) {
        if (a[p1].b <= a[p2].b) {
            t.update(a[p1].c, a[p1].w);
            tmp[cur++] = a[p1++];
        }
        else {
            a[p2].ans += t.query(a[p2].c);
            tmp[cur++] = a[p2++];
        }
    }
    while (p1 <= mid) {
        t.update(a[p1].c, a[p1].w);
        tmp[cur++] = a[p1++];
    }
    while (p2 <= r) {
        a[p2].ans += t.query(a[p2].c);
        tmp[cur++] = a[p2++];
    }
    for (int i = l; i <= mid; i++) t.update(a[i].c, -a[i].w);
    for (int i = l; i <= r; i++) a[i] = tmp[i];
};

```

9.3 ST 表

```

struct ST {
    int st[LOGN][MAXN];
    void build(int n, int *a) {
        int t = std::__lg(n) + 1;
        for (int i = 1; i <= n; i++) st[0][i] = a[i];
        for (int i = 1; i <= t; i++) {
            for (int j = 1; j <= n; j++) {
                st[i][j] = std::max(st[i - 1][j], st[i - 1][j + (1 << (i - 1))]);
            }
        }
    }
    int query(int l, int r) {
        int t = std::__lg(r - l + 1);
        return std::max(st[t][l], st[t][r - (1 << t) + 1]);
    }
};

```

9.4 树链剖分

```

auto dfs1 = [&](auto self, int u, int f) -> void {
    siz[u] = 1; fa[u] = f; dep[u] = dep[f] + 1;
    for (const auto &v : G[u]) {
        if (v == f) continue;
        self(self, v, u);
        siz[u] += siz[v];
        if (siz[v] > siz[son[u]]) son[u] = v;
    }
};

```

```

    }
};
auto dfs2 = [&](auto self, int u, int tp) -> void {
    top[u] = tp;
    if (son[u]) self(self, son[u], tp);
    for (const auto &v : G[u]) {
        if (v == son[u] || v == fa[u]) continue;
        self(self, v, v);
    }
};
auto lca = [&](int u, int v) {
    while (top[u] != top[v]) {
        if (dep[top[u]] < dep[top[v]]) std::swap(u, v);
        u = fa[top[u]];
    }
    if (dep[u] > dep[v]) return v;
    return u;
};

```

9.5 树上倍增 / RMQ LCA (原模板)

```

struct St {
    int st[MAXN][25];
    int dfnmin(int x, int y) {
        if (dfn[x] < dfn[y]) return x;
        return y;
    }
    void init() {
        int t = std::__lg(n) + 1;
        for (int i = 1; i <= n; i++) st[dfn[i]][0] = fa[i];
        for (int j = 1; j <= t; j++) {
            for (int i = 1; i + (1 << (j - 1)) <= n; i++) {
                st[i][j] = dfnmin(st[i][j - 1], st[i + (1 << (j - 1))][j - 1]);
            }
        }
    }
    int query(int l, int r) {
        int t = std::__lg(r - l + 1);
        return dfnmin(st[l][t], st[r - (1 << t) + 1][t]);
    }
}st;

auto lca = [&](int x, int y) {
    if (x == y) return x;
    if (dfn[x] > dfn[y]) std::swap(x, y);
    return st.query(dfn[x] + 1, dfn[y]);
};

```

10. 图论

10.1 最短路 (Dijkstra)

```

auto dijkstra = [&](int s) {
    std::priority_queue <std::pair<int, int>, std::vector<std::pair<int, int> >,
        ⇨ std::greater<std::pair<int, int> > > q;
    std::fill(dis.begin(), dis.end(), INF);

```

```

dis[s] = 0;
q.push({0, s});
while (q.size()) {
    int u = q.top().second; q.pop();
    if (vis[u]) continue;
    vis[u] = 1;
    for (const auto &p : G[u]) {
        int v = p.first, w = p.second;
        if (dis[v] > dis[u] + w) {
            dis[v] = dis[u] + w;
            q.push({dis[v], v});
        }
    }
}
};

```

10.2 Dinic 最大流

```

struct Flow {
    int n;
    struct Edge {
        int to, w, rev;
    };
    std::vector <std::vector<Edge> > G;
    std::vector <int> dep, cur;
    Flow(int _n = 0) : n(_n) {
        G.resize(n + 1);
        dep.resize(n + 1);
        cur.resize(n + 1);
    }
    void add(int u, int v, int w) {
        int a = G[u].size(), b = G[v].size();
        G[u].push_back({v, w, b});
        G[v].push_back({u, 0, a});
    }
    bool bfs(int s, int t) {
        for (int i = 0; i <= n; i++) {
            dep[i] = 0;
            cur[i] = 0;
        }
        std::queue <int> q;
        q.push(s); dep[s] = 1;
        while(q.size()) {
            int u = q.front(); q.pop();
            for (const auto &[v, w, id] : G[u]) {
                if (dep[v] || !w) continue;
                dep[v] = dep[u] + 1;
                q.push(v);
            }
        }
        return (dep[t] != 0);
    }
    static constexpr int INF = 0x7f7f7f7f;
    int dfs(int u, int t, int flow) {
        if (u == t) return flow;
        int rest = flow;
        for (int i = cur[u]; i < G[u].size(); i++) {
            const auto &[v, w, rev] = G[u][i];
            if (!w || dep[v] != dep[u] + 1) continue;
            int used = dfs(v, t, std::min(w, rest));

```

```

        rest -= used;
        G[u][i].w -= used; G[v][rev].w += used;
        cur[u] = i;
        if (!rest) return flow;
    }
    return flow - rest;
}
ll dinic(int s, int t) {
    ll ret = 0;
    while (bfs(s, t)) {
        int flow;
        while ((flow = dfs(s, t, INF))) ret += flow;
    }
    return ret;
}
};

```

10.3 费用流 (MCMF)

```

struct Flow {
    static constexpr int INF = 0x7f7f7f7f;
    int n;
    struct Edge {
        int to, w, rev, c;
    };
    std::vector<std::vector<Edge>> G;
    std::vector<int> dep, cur, dis, inq;
    Flow(int _n = 0) : n(_n) {
        G.resize(n + 1);
        dep.resize(n + 1);
        cur.resize(n + 1);
        dis.resize(n + 1);
        inq.resize(n + 1);
    }
    void clear(int _n) {
        for (int i = 0; i <= n; i++) G[i].clear();
        G.clear(); G.resize(_n + 1);
        dep.clear(); dep.resize(_n + 1, 0);
        cur.clear(); cur.resize(_n + 1, 0);
        dis.clear(); dis.resize(_n + 1, 0);
        inq.clear(); inq.resize(_n + 1, 0);
        n = _n;
    }
    void add(int u, int v, int w, int c) {
        int a = G[u].size(), b = G[v].size();
        G[u].push_back({v, w, b, c});
        G[v].push_back({u, 0, a, -c});
    }
    bool SPFA(int s, int t) {
        for (int i = 0; i <= n; i++) {
            dep[i] = 0;
            cur[i] = 0;
            inq[i] = 0;
            dis[i] = INF;
        }
        std::queue<int> q;
        q.push(s); dep[s] = 1; inq[s] = 1; dis[s] = 0;
        while(q.size()) {
            int u = q.front(); q.pop(); inq[u] = 0;
            for (const auto &[v, w, id, c] : G[u]) {

```

```

        if (w && dis[v] > dis[u] + c) {
            dis[v] = dis[u] + c;
            dep[v] = dep[u] + 1;
            if (!inq[v]) {
                q.push(v);
                inq[v] = 1;
            }
        }
    }
}
return (dep[t] != 0);
}
ll mincost = 0;
int dfs(int u, int t, int flow) {
    if (u == t) return flow;
    int rest = flow;
    for (int i = cur[u]; i < G[u].size(); i++) {
        const auto &[v, w, rev, c] = G[u][i];
        if (!w || dep[v] != dep[u] + 1 || dis[v] != dis[u] + c) continue;
        int used = dfs(v, t, std::min(w, rest));
        rest -= used;
        G[u][i].w -= used; G[v][rev].w += used;
        mincost += 1ll * c * used;
        cur[u] = i;
        if (!rest) return flow;
    }
    return flow - rest;
}
std::pair<ll, ll> MCMF(int s, int t) {
    ll ret = 0;
    while (SPFA(s, t)) {
        int flow;
        while ((flow = dfs(s, t, INF))) ret += flow;
    }
    return std::make_pair(ret, mincost);
}
};

```

10.4 Tarjan 强连通分量

```

auto tarjan = [&](int u, auto self) -> void {
    dfn[u] = low[u] = ++dcnt;
    s.push(u); ins[u] = 1;
    for (const auto &v : G[u]) {
        if (!dfn[v]) {
            self(v, self);
            low[u] = std::min(low[u], low[v]);
        }
        else if (ins[v]) low[u] = std::min(low[u], dfn[v]);
    }
    if (dfn[u] == low[u]) {
        ++scc;
        int cur;
        do {
            cur = s.top(); s.pop(); ins[cur] = 0;
            belong[cur] = scc;
        } while (cur != u);
    }
}
};

```